第一节上机实验--SVD及其应用

一、 SVD算法实现

基于给定的两组正定矩阵和拟对角阵Q_5 ，Q_4，Σ，组装矩阵A

$$Q_5 = \begin{bmatrix} +0.130739 & +0.608762 & -0.043819 & +0.780233 & -0.040407 \\ +0.842723 & -0.093658 & -0.312899 & -0.107165 & -0.414313 \\ -0.251167 & -0.638035 & -0.511983 & +0.505413 & -0.110713 \\ +0.426363 & -0.217889 & +0.017690 & +0.144392 & +0.865781 \\ +0.166897 & -0.407521 & +0.798582 & +0.321653 & -0.254711 \end{bmatrix}$$

$$Q_4 = \begin{bmatrix} -0.690420 & -0.161962 & +0.440611 & +0.550410 \\ -0.657117 & +0.395218 & -0.090602 & -0.635446 \\ +0.008191 & +0.773622 & -0.356826 & +0.523562 \\ +0.302408 & +0.468060 & +0.818736 & -0.138346 \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} 100 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 0.001 \\ 0 & 0 & 0 & 0 \end{bmatrix}_{5\times 4}$$

$$A = \begin{bmatrix} -13.026541 & +0.285432 & +5.211142 & +3.325243 \\ -57.568112 & -14.043353 & +37.227220 & +46.963122 \\ +21.533410 & +1.506957 & -10.469915 & -9.796977 \\ -28.005090 & -7.765174 & +18.982906 & +24.852922 \\ -8.844273 & -4.251769 & +7.694661 & +11.817533 \end{bmatrix}$$

1、对A做SVD，并逐项分析它的秩，奇异值，对应右特征向量和左特征向量，以及秩一矩阵之和形式。

2、分析取不同的k时，Ak对应的能量比值。

二、SVD应用于图像压缩

Task 1：灰度图截断重建（核心代码，10 分钟）

- 目标：写出三行核心代码并跑通
- 步骤：

    1. 取一个通道（或转灰度）得矩阵 A ∈ R^{H×W}

2. `U, s, Vt = np.linalg.svd(A, full_matrices=False)`（强调 `full_matrices=False` 返回约化形式）

3. `A_k = (U[:, :k] * s[:k]) @ Vt[:k, :]`

- 预期输出：给定 k=20，打印 A_k 的形状并显示重建图
- 易错点强调：输入必须 float，重建后 `np.clip(A_k, 0, 1)`

Task 2：RGB 三通道与可视化网格（10 分钟）

- 目标：对三个通道各自做 SVD，拼出 k 网格对比图
- 步骤：

1. 对 R/G/B 分别计算 SVD（`svd_ch = [np.linalg.svd(img[:,:,c], full_matrices=False) for c in range(3)]`）

2. 选 k ∈ {1, 2, 3, 5, 8, 12, 20, 35, 50, 80, 120}，用 `subplots` 摆成"原图 + 各 k"网格

- 预期输出：一张 3×4 或 4×3 网格图，肉眼可见 k 从小到大"由色块到清晰"的渐变

Task 3：定量指标（10 分钟）

- 目标：实现一个 `stats(k)` 函数，输出压缩比、相对误差、PSNR
- 关键技巧（本课亮点）：**不重建、不逐像素相减**，直接用酉不变性算误差
  - 相对误差 = √(Σ_{i>k} σ??) / √(Σ? σ??)（三通道求和后合计）
  - MSE = Σ_{i>k} σ?? / (H·W·C)；PSNR = 10·log??(1/MSE)（满秩时 MSE=0，PSNR = ∞）
  - 参数量 = C·k·(H+W+1)（每通道 U 占 Hk、Σ 占 k、V? 占 kW；注意 +1 是奇异值本身）
  - 压缩比 = (H·W·C) / [C·k·(H+W+1)] = HW / [k(H+W+1)]

- 预期输出：k 从 1 到 min(H,W) 的一张指标表（可用教师参考表核验，见附录）

Task 4：三条曲线与盈亏平衡点（10 分钟）

- 目标：把指标画成曲线并解读

    1. **Scree plot**：σ? 随 i 的衰减（y 轴 log 刻度）

    2. **误差/PSNR vs k**：相对误差下降曲线（或 PSNR 上升曲线）

    3. **压缩比 vs k**：单调下降曲线，并画一条竖虚线标注盈亏平衡点
       k* = HW/(H+W+1)（620×820 时 ≈ 353）

- 预期输出：三张图 + 一句话结论——k 超过 k* 后"越压越大"

Task 5：三个关键发现（约 20 分钟）

围绕记录中的三条发现组织讨论，先让学生观察自己的实验数据再揭示：

发现 ① 能量陷阱（Energy Trap）

- 提问：计算能量占比 E(k) = Σ_{i≤k}σ?? / Σ?σ??。k=1 是多少？k=4 是多少？图看着怎样？
- 教学点：评估质量看 PSNR 与眼睛，**不要只看能量占比**。

发现 ② 盈亏平衡点（Break-even Point）

- 提问：压缩比什么时候跌破 1？给出推导。
-

发现 ③ 高频细节最先牺牲（High-Frequency Details Go First）

- 提问：k 很小时，图的哪部分先失真？
-

三、分组讨论与提高

实战 1：Eigenfaces 人脸识别（压缩 + 识别一体）
出处：M. Turk, A. Pentland, "Eigenfaces for Recognition", Journal of Cognitive Neuroscience 3(1):71–86, 1991. DOI: 10.1162/jocn.1991.3.1.71 (https://doi.org/10.1162/jocn.1991.3.1.71)（约 13,775 次引用）。
机制（论文原文逻辑）：

1. 取 N 张对齐人脸，每张展开为 HW 维向量（如 128×128 = 16,384 维）

2. 求协方差矩阵的主成分 → 特征向量就是 eigenfaces（一张张"鬼脸"基图），保留前 k（M）个最大特征值对应的
3. 每张脸 = 均值脸 + $\sum$ 权重 × eigenface（压缩：每张脸只需 k 个权重）
4. 识别 = 把新脸投影到"脸空间"，在系数空间做最近邻分类（识别：直接比对 k 维向量）
   关键来源：Turk & Pentland 明言其思想来自 Sirovich & Kirby (1987)、Kirby & Sirovich (1990) 的 "eigenpictures" 压缩工作——"存少量权重 + 少量标准图（eigenpicture）即可重构整个脸图集"，这正是共享基压缩的原始表述。
   生产基线（真实落地）：OpenCV 至今随附 cv::face::EigenFaceRecognizer，官方文档称"保留 80 个分量几乎总是足够"，在 AT&T 人脸库上识别率约 96%。—— 这是"共享基 SVD/PCA"在工业界仍作为标准基线的直接证据。
   压缩比：库越大越划算（$HW\gg N$ 时 $\approx N/k$）。

实战 2：3D Morphable Model（3D 脸的系数化编码）
出处：V. Blanz, T. Vetter, "A Morphable Model for the Synthesis of 3D Faces", SIGGRAPH '99, pp. 187–194, 1999. DOI: 10.1145/311535.311556 (https://doi.org/10.1145/311535.311556)（约 16,001 次引用）。
机制（论文原文逻辑）：

1. 用 200 张 3D 扫描人脸建立对应（每张脸：形状向量 $S \in \mathbb{R}^{3n}$ 存 n 个顶点 XYZ，纹理向量 $T \in \mathbb{R}^{3n}$ 存顶点 RGB）
2. 对形状、纹理协方差矩阵各做一次 PCA → S_{\text{model}} = \bar S + \sum_i \alpha_i s_i$，$T_{\text{model}} = \bar T + \sum_i \beta_i t_i
3. 任意一张 3D 脸 = 均值 + 少量系数 (α,β)，系数服从多元正态，还可约束"像不像脸"
   压缩：一张含数万顶点（$3n$ 个坐标，约 21 万数）的 3D 脸，被 $k \approx 100$ 个系数（形状+纹理）刻画 → 单脸约 2100×；基由 200 张训练集一次性学得后摊薄。
   现代生产：这是当今人脸动画、游戏虚拟形象、视频通话 avatar、单图 3D 重建、医学/法医面部分析的底层技术（3DMM 及其后代 FaceGen、LDM 等均沿用"系数化 + 共享基"范式）。

实战3：边缘设备 → 云端的低带宽上传（顺带隐私）

场景：4G/NB-IoT 门禁机抓拍后回传，流量按 MB 计费。

数字：原图 4096 B/帧 → 32 个 int8 系数 = 32 B，约 128x。基（256 KB）只在设备激活或换版时同步一次，之后每帧只发系数。

这里有个额外红利：隐私。上传的是 k 维投影，不是像素；云端拿到系数，没有基也还原不出人脸。而且可以有选择地丢弃前几个"光照/增益维"（对齐后前几个主成分通常是全局亮度），进一步去标识化。

坑，必须处理

两端基必须字节级一致 → 握手时交换基 ID + 版本，不匹配就触发基下发。
量化参数（lo/scale）要随基一起下发，否则端点解码出的系数是错的。
per-dimension 量化 + 分位截断：Eigenfaces 各维系数量纲差异极大，全局量化必炸；用 min-max 又会被离群点拉爆区间，取 0.1%~~99.9% 分位。实测 int8 相对 fp32 只掉 0.01~~0.04 dB，最大像素差 0.0015 —— 量化基本是免费午餐。
对齐用的 (μ, σ) 要么传（2 个 float16），要么在编码端固定归一化。

实战4：连拍/视频序列的时序共享基

场景：一个摄像头连续抓拍同一批人，或在跑一段人脸视频。把帧当成样本维，序列共享一组基。

这时发生了质变：每张图的 k 个系数，在时间轴上变成 k 条平滑的一维轨迹。身份变化慢、光照变化慢，轨迹的可预测性极强：

对每条轨迹做 delta 编码 + 线性预测，再叠 RLE/熵编码，实测还能再压 2~3x
对比逐帧 JPEG：JPEG 每帧独立，帧间冗余一点没用上；而共享基天然把帧间相关吃掉了
序列越长，基的 256 KB 摊得越薄 → 这是唯一一个"压得越久越划算"的方案

关键设计

定期插 I 帧重算基（比如每 300 帧或姿态漂移超阈值），防止基过时导致重建糊掉。
场景/人员切换检测 → 触发新基，而不是硬扛。